Pr. I. HAMZAOUI / Filière : CCN-2
Module : Machine Learning
Année universitaire 2025/2026

Travaux Pratiques N°9

Nom et prénom : ……

Objectifs du TP :

- Utiliser GMM et DBSCAN pour segmenter les données clients ;
- Comparer les trois métriques de qualité (Silhouette, DBI, CH) pour les deux techniques.
- Étudier l'impact des points extrêmes sur les deux techniques clustering
  Choisir la meilleure configuration (k pour GMM, eps/min_samples pour DBSCAN) en s'appuyant à la fois sur : les métriques, la visualisation et l'interprétation métier des clusters.

Livrables à rendre : Complétez ce document manuscritement par le travail demandé.

Données : Vous disposez du fichier **Dataset_TP9.csv.** Ce dataset contient 986 clients décrits par deux variables continues :

- **Recency** : nombre de jours écoulés depuis la dernière commande (0 = client très récent, valeur élevée = client inactif depuis longtemps) ;
- **Monetary** : montant total dépensé par le client (en MAD).

1) Exécutez le code ci-dessous et visualisez les données brutes des clients

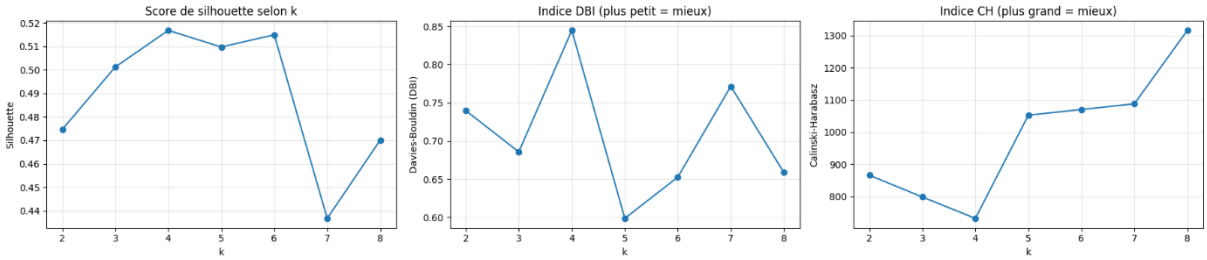
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.mixture import GaussianMixture
from sklearn.cluster import DBSCAN
from sklearn.metrics import (
    silhouette_score,
    davies_bouldin_score,
    calinski_harabasz_score)
```

```
df = pd.read_csv("Dataset_TP9.csv")
```

```
plt.figure(figsize=(7, 5))
plt.scatter(df["Recency"], df["Monetary"], s=15, alpha=0.6)
plt.xlabel("Recency (jours depuis la dernière commande)")
plt.ylabel("Monetary (MAD dépensés)")
plt.title("Nuage de points des clients (Recency vs Monetary)")
plt.grid(True, alpha=0.3)
plt.show()
```

```
X = df[["Recency", "Monetary"]].values
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

2) Après avoir entraîné un modèle GaussianMixture sur les données précédentes, on obtient les courbes suivantes pour les trois métriques (Silhouette, DBI, Calinski–Harabasz) en fonction de k:



En vous appuyant sur ces trois courbes, proposez un nombre de composantes optimal k pour le GMM. Entraînez un modèle GaussianMixture avec ce K sur les données standardisées X_scaled puis récupérez les labels de cluster prédits (labels_gmm).

```
best_k =    # à adapter selon les résultats interprétés
gmm_final = GaussianMixture(


    random_state=0
)
labels_gmm =
```

3) Exécutez le code ci-dessous pour visualiser les clusters GMM pour le k choisi.

```
# Visualisation finale GMM
plt.figure(figsize=(7, 6))
scatter = plt.scatter(
    X_scaled[:, 0],
    X_scaled[:, 1],
    c=labels_gmm,
    s=20,
    cmap="tab10",
    alpha=0.8
)
plt.xlabel("Recency (standardisé)")
plt.ylabel("Monetary (standardisé)")
plt.title(f"GMM (k={best_k}) - clusters sur données standardisées")
plt.grid(True, alpha=0.3)
plt.legend(*scatter.legend_elements(), title="Clusters")
plt.show()
```

4) Pour tester un modèle clustering DBSCAN. Choisissez des valeurs des hyperparamètres eps et min_samples de votre choix.

```
configs = [
    (   ,   ),
    (   ,   ),
    (   ,   ),
    (   ,   ),
    (   ,   ),
    (   ,   ),
]
```

5) En fonction des configurations que vous avez choisi, exécutez le code ci-dessous et complétez le tableau demandé.

```
resultats_dbscan = []

for eps, min_samples in configs:
    dbscan = DBSCAN(eps=eps, min_samples=min_samples)
    labels_db = dbscan.fit_predict(X_scaled)

    unique_labels = np.unique(labels_db)
    n_noise = np.sum(labels_db == -1)
    n_clusters = len(unique_labels[unique_labels != -1])

    mask_clusters = labels_db != -1
    if n_clusters > 1 and mask_clusters.sum() > n_clusters:
        sil = silhouette_score(X_scaled[mask_clusters], labels_db[mask_clusters])
        dbi = davies_bouldin_score(X_scaled[mask_clusters], labels_db[mask_clusters])
        ch  = calinski_harabasz_score(X_scaled[mask_clusters], labels_db[mask_clusters])
    else:
        sil = dbi = ch = np.nan

    # On ajoute un résultat par config
    resultats_dbscan.append({
        "eps": eps,
        "min_samples": min_samples,
        "n_clusters": n_clusters,
        "n_noise": n_noise,
        "Silhouette": sil,
        "DBI": dbi,
        "Calinski_Harabasz": ch
    })
```

```
# DataFrame
df_scores_dbscan = pd.DataFrame(resultats_dbscan)
print("\nRésultats DBSCAN pour différentes configurations :")
print(df_scores_dbscan)
```

eps	min_samples	n_clusters	n_noise	silhouette	DBI	CH

6) Après avoir analyser les résultats précédents. Choisissez une ou deux meilleures configurations qui respectent à la fois un nombre de bruits respectables, des métriques convenables et un nombre de cluster correctement interprétable.

```
eps_best =
min_samples_best =
dbscan_best = DBSCAN(eps=eps_best, min_samples=min_samples_best)
labels_best = dbscan_best.fit_predict(X_scaled)
```

```
X_orig = scaler.inverse_transform(X_scaled)
```

```python
unique_labels = np.unique(labels_best)
colors = plt.cm.tab10(np.linspace(0, 1, len(unique_labels)))
plt.figure(figsize=(7, 6))
for lab, col in zip(unique_labels, colors):
    mask = (labels_best == lab)
    if lab == -1:
        # Bruit / valeurs extrêmes
        plt.scatter(
            X_orig[mask, 0],
            X_orig[mask, 1],
            c=[col],
            marker="x",
            s=80,
            label="Bruit / valeurs extrêmes" )
    else:
        # Clusters normaux
        plt.scatter(
            X_orig[mask, 0],
            X_orig[mask, 1],
            c=[col],
            s=25,
            alpha=0.8,
            label=f"Cluster {lab}" )
plt.xlabel("Recency (jours)")
plt.ylabel("Monetary (MAD)")
plt.title(f"DBSCAN sur Recency/Monetary\n(eps={eps_best}, min_samples={min_samples_best})")
plt.grid(True, alpha=0.3)
plt.legend()
plt.show()
```

Après avoir exécuter le code ci-dessus, que pensez-vous des clusters générés par DBSCAN ? Est-ce qu'ils sont les mêmes que ceux générés par le GMM ? Si ce n'est pas le cas, expliquez pourquoi ?

Réponse :